

DataStorm 7.0 - Storming Round

Estimating January 2026 Maximum Monthly Purchase Potential for 20,000 Traditional-Trade Beverage Outlets in Sri Lanka

Team: DataX

Approach Summary

The competition target - "Maximum Monthly Purchase Potential" - has no observed ground truth. Historical volume V is censored: $V = \min(D, C)$, where D is true demand and C is systemic constraint (credit, stockouts, delivery caps). We therefore build a defensible statistical framework rather than train a supervised model. Each outlet's January 2026 potential is estimated as the maximum of (a) its own 90th-percentile monthly volume across unconstrained months and (b) the 90th-percentile monthly volume of its peer cohort (Outlet_Type $\times$ Size $\times$ Province $\times$ POI-density-tier, with hierarchical fallback when a cohort is sparse). We apply distributor-specific seasonality multipliers (calibrated empirically from the categorical Seasonality_Index), a year-over-year growth factor, and a holiday-count adjustment for January 2026. A log-linear regression on the unconstrained subset cross-checks the peer-quantile estimate (Spearman $\rho = 0.889$); a 0.6/0.4 blend produces the final number.

Pipeline

Bronze (raw, immutable, sha256 manifest) $\rightarrow$ Silver (cleaned + quarantine with documented rejection reasons) $\rightarrow$ POI enrichment (16 OSM tags $\times$ Sri Lanka bbox via Overpass API + BallTree spatial join) $\rightarrow$ Gold (outlet-month panel + 50 outlet-level features) $\rightarrow$ Model (constraint detection + peer-quantile ceiling + cross-check + internal-consistency validation) $\rightarrow$ Predictions CSV.

Key Numbers

Raw transaction rows ingested	2,376,389
Rows quarantined with reason	73,485 (3.09%)
POIs scraped from OpenStreetMap	32,246 across 16 tag types
Outlets covered by predictions	20,000 (100%)
Cross-check Spearman ρ (peer-Q90 vs log-linear)	0.889
Final blend weights	0.6 peer-Q90 + 0.4 log-linear
Predicted potential (median / mean)	181.0 L / 306.8 L

a. Data Forensics and Hygiene

All five raw files are ingested into the Bronze layer with sha256 manifests, then subjected to five reusable parameterised data-quality checks plus a forensics layer that targets legacy SFA / distributor-ERP artifacts. Records failing any check are quarantined with a documented `\_rejection\_reason` - none are silently dropped.

DQ Architecture (reusable & parameterised)

`check_duplicates(df, primary_keys)`, `check_nulls(df, mandatory_columns)`, `check_referential_integrity(df, fk_column, valid_values)`, `check_value_range(df, column, min_val, max_val)`, `check_format(df, column, validator)`. Wrapped by `apply_check()` which persists rejects to `data/silver/quarantine/{dataset}_rejected.parquet` and appends a row to `_quarantine_summary.csv`.

Quarantine Summary (by dataset)

Dataset	In	Passed	Rejected	Retention %
holiday	349	151	198	43.2700
outlet_coordinates	20000	19564	436	97.8200
outlet_master	20000	19804	196	99.0200
seasonality	360	360	0	100.0000
transactions	2376389	2303734	72655	96.9400

Named Legacy-SFA Artifacts Caught

Finding	Count	Treatment
Cross-file Outlet_ID integrity	0	reported_clean
Outlet_Type typos normalised	585	cleaned
Transaction value classification	0	cleaned_and_flagged
Codebook references absent column	1	flagged
Codebook filename divergence	1	flagged
Automated-entry signatures	0	none_found
Stockout months (zero sandwiched between non-zero)	81165	flagged_for_modeling
Dead-then-resurrected outlets	2704	flagged

Highlights: 61,909 duplicate transactions on (Outlet_ID, Year, Month, SKU_ID); 240 outlets with coordinates outside the Sri Lanka bounding box; 585 Outlet_Type entries normalised from typos such as 'Grocery' → 'Grocery'; 4,611 return transactions identified via a 3-way classification of (Volume_Liters, Total_Bill_Value) sign pairs (kept and net-subtracted, not silently dropped). The supplied codebook references a Product_Name column absent from the transactions file and names the seasonality file as `distributor_seasonality.csv` while the actual file is `distributor_seasonality_details.csv` - both logged as governance findings.

b. POI Data Acquisition

Point-of-Interest data is not provided. We acquire it from OpenStreetMap via the public Overpass API. A naive per-outlet loop (20,000 outlets $\times$ ≥ 1 sec rate-limited request) would take ≥ 5.5 hours; we instead issue one bbox-query per POI tag covering all of Sri Lanka (5.9°-9.9° N, 79.5°-82.0° E), yielding 16 queries total. Raw JSON responses are cached to data/bronze/poi_raw/ so re-runs are free, and an exponential-backoff retry loop handles HTTP 429/504 rate-limit responses.

POI Tags Queried (FMCG-beverage catchment logic)

Footfall: amenity=school, university, hospital, marketplace; highway=bus_stop. Retail proxy: shop=supermarket, convenience, alcohol; amenity=pharmacy, bank. Dining/tourism: amenity=restaurant; tourism=hotel, attraction. Community/transit: amenity=place_of_worship, fuel; leisure=park.

Per-Outlet Mapping (spatial join)

All POI coordinates are loaded into a sklearn BallTree with haversine metric (radians). For each outlet we query count_only=True at three rings - 1 km, 2 km, 5 km - producing 16 tags $\times$ 3 radii = 48 POI-density features per outlet. Aggregate poi_total_{r}km columns supply cohort-tier signal for the modelling step.

POI Inventory (Sri Lanka, all 16 tags)

Tag	POI_count_in_SL
amenity=place_of_worship	9284
amenity=school	5029
highway=bus_stop	3437
amenity=restaurant	3212
tourism=hotel	2958
amenity=bank	2225
shop=supermarket	1140
shop=convenience	1033
amenity=fuel	952
amenity=hospital	891
leisure=park	515
amenity=pharmacy	485
tourism=attraction	403
amenity=marketplace	335
shop=alcohol	203
amenity=university	144

Limitation acknowledged: OpenStreetMap coverage is uneven across Sri Lanka - Western Province (Colombo metro) is densely tagged whereas North-Western and Southern rural regions are sparser. We mitigate this by computing the POI-density tier within province (per-quartile bucket using poi_total_2km), so the modelling cohort uses relative density rather than absolute counts.

c. Causal Base Logic - Uncapping the Censored Demand

Observed monthly volume $V(i,t)$ for outlet i in month t is the minimum of two unobserved variables: $V(i,t) = \min(D(i,t), C(i,t))$, where D is latent consumer demand and C is systemic constraint (credit limit, stockout, delivery cap). The objective - "Maximum Monthly Purchase Potential" - is D under the assumption that constraints are relaxed.

Framework (4 steps)

Step A - Constraint detection (OR of three rules): (i) zero-volume month with positive neighbours (stockout signature); (ii) zero-volume month in an outlet that has any positive month (intermittent supply); (iii) Cooler_Count = 0 with zero_months_share > 0.30 (infrastructure-limited).

Step B - Peer-conditional Q90 ceiling. Group outlets by (Outlet_Type × Size × Province × poi_tier) and compute the 90th-percentile monthly volume across unconstrained outlet-months. Hierarchical fallback (Type×Size×Province×Tier → Type×Size×Province → Type×Size → Type → Global) when a cohort has fewer than 30 unconstrained observations. Each outlet's base potential = max(own Q90 of unconstrained months, peer-cohort Q90).

Step C - Projection: $\hat{P}(\text{Jan } 2026) = \text{base} \times m(\text{distributor, Jan } 2026) \times g(\text{distributor}) \times h(\text{Jan } 2026)$. m is the distributor-specific seasonality multiplier calibrated empirically from the categorical Seasonality_Index (e.g., 'Favorable' → 1.30-1.43× baseline; 'Un-Favorable' → 0.59-0.65×). g is the geometric mean of Jan-over-Jan growth per distributor, clipped to [0.85, 1.30]. h is a holiday-count adjustment that compares 2026 January's known fixed/lunar holidays (Duruthu Poya, Tamil Thai Pongal) against the historical Jan average. Sanity bounds: floor = Q95 of own history; ceiling = 5 × peer cohort Q90.

Step D - Cross-check via log-linear regression on the unconstrained subset. We fit OLS on $\log(1 + \text{monthly_volume})$ using outlet attributes (Type, Size, Province, POI density, Cooler_Count, coordinates) only on unconstrained outlet-months - a complete-case censored-regression proxy (Tobin 1958; Greene 2008). Predictions are exp-transformed and projected with the same Jan 2026 multipliers, then compared to peer-Q90 by Spearman ρ . When $\rho \geq 0.75$ we blend final = 0.6·peer-Q90 + 0.4·log-linear; otherwise peer-Q90 alone (with honest disclosure in this report).

Internal Consistency (no held-out y, so we validate stability)

Check	Result
Constraint share > 25% (outlet count)	13,847 of 19,564 outlets
Magnitude ratio median (predicted / observed mean)	2.81
Quantile sensitivity figure	outputs/audit/sensitivity_quantile.png
Spatial Spearman ρ (predicted vs poi_total_2km)	0.034 (p=2.29e-06)
Cross-method Spearman ρ (peer-Q90 vs log-linear)	0.889
Peer-fallback Level-0 outlet share	99.92% (19,548 / 19,564)

References: Tobin 1958 (censored regression); Koenker & Bassett 1978 (quantile regression); Buchinsky 1998 (recent quantile-regression developments); Greene 2008, Econometric Analysis, chap. 19.

Acknowledged Limitations

(1) Constraint-rule (ii) is partially circular: outlet activity is used to identify constraint, and the Q90 over the resulting unconstrained subset feeds back into the same outlet's potential. We mitigate by combining own-Q90 with peer-Q90 in a max operator. (2) OpenStreetMap rural coverage is uneven; within-province POI-tier normalisation reduces but does not eliminate the bias. (3) Spatial correlation between predicted potential and total POI density is weak (Spearman $\rho = 0.034$) because POI signal is already absorbed by the cohort grouping. (4) The Seasonality_Index for January 2026 is extrapolated from 2023-2025 Januaries via majority vote per distributor.

d. GenAI Transparency Log

Every design decision in this submission came from the team. We chose statistical estimation over supervised ML once we saw the problem statement clearly rules out a target variable. We laid out the Bronze, Silver and Gold layers ourselves because the rubric calls for it explicitly. We defined the peer cohort, wrote the constraint rules, picked the sanity bounds, and decided what limitations to surface honestly in this report. The AI assistant (Claude Code) helped us move faster on specified tasks: writing boilerplate code, debugging tracebacks, and looking up the right citation when we needed one. Every substantive interaction is logged in docs/genai_log.md with the prompt, the output, what we did with it, and how we checked it.

Representative Entries

#	Phase	Prompt	Action
1	Planning	"Survey actual data schemas (column names, row counts, sample rows) for all 5 pr	Accepted with verification. Cross-checked each claim by reading the raw files independentl
2	Planning	"Is supervised ML training appropriate when the problem statement says we are no	Accepted.
3	Planning	"Practical sanity check across data + methodology + technical execution before c	Five concerns merged into plan; three acknowledged as PDF limitations or implementation-ti
4	Phase 1	"Write reusable parameterised check_duplicates that returns (clean, rejected) an	Accepted as-is. Keeps signal for the audit trail.
5	Phase 1	"Bronze copy must compute sha256 + introspect row/column counts without modifyin	Accepted as-is.
6	Phase 2	"Detect automated-entry signatures via run-length encoding of identical Volume_L	Accepted with caveat. Worked correctly but did not catch any signatures in this dataset (n
7	Phase 2	"PK for holiday should be just Date"	REJECTED after first run. 98.85% of holiday rows were flagged as duplicates because same d
8	Phase 2	"Normalise Outlet_Type typos and log count of corrections"	REJECTED after first run. Reported 5,958 corrections when actual changes were 585 (it incl

Where We Pushed Back on the AI

Three cases stand out. First, the AI initially suggested using Date alone as the primary key for the holiday list. When we tried that, almost 99% of rows were flagged as duplicates, because the same date legitimately appears with Public, Bank and Mercantile categories. We changed the key to (Date, Holiday_Type) and the duplicate rate fell to a sensible 56.73%, which we found are real repeated entries left over from the legacy export. Second, the typo counter for Outlet_Type was over-reporting: it counted every row whose post-normalisation value happened to match a canonical label, including rows that were already correct. We fixed the logic to count only rows whose value actually changed, which brought the number from a misleading 5,958 down to the true 585. Third, the AI's first take on negative bill values was to quarantine them all as errors. We pushed back because some of those rows are real returns. We replaced the rule with a three-way classification that keeps returns and free-of-charge promos as signal, and quarantines only the impossible combinations. That preserved 4,611 return rows that the simpler approach would have thrown away.

Where the AI Genuinely Saved Us Time

Three areas. The reusable parameterised DQ functions in src/dq_checks.py were drafted by the AI once we had decided on the five check types, which saved us maybe two hours of boilerplate. The Overpass API client with retry-with-backoff and JSON caching, and the BallTree spatial join with haversine in radians, came together quickly because the AI knew the right idioms. And during a tricky moment in Phase 5 the AI surfaced the right set of references (Tobit 1958, Buchinsky 1998, Koenker and Bassett 1978) so we could frame the causal logic on solid ground. We checked the citations and the convergence numbers ourselves before they went into this report.